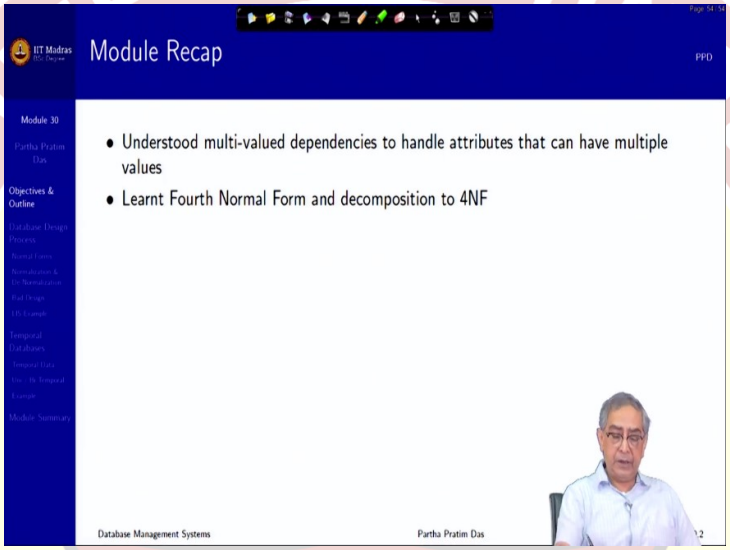
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pritam Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Relational Database design/10:
Design Summary and Temporal Data

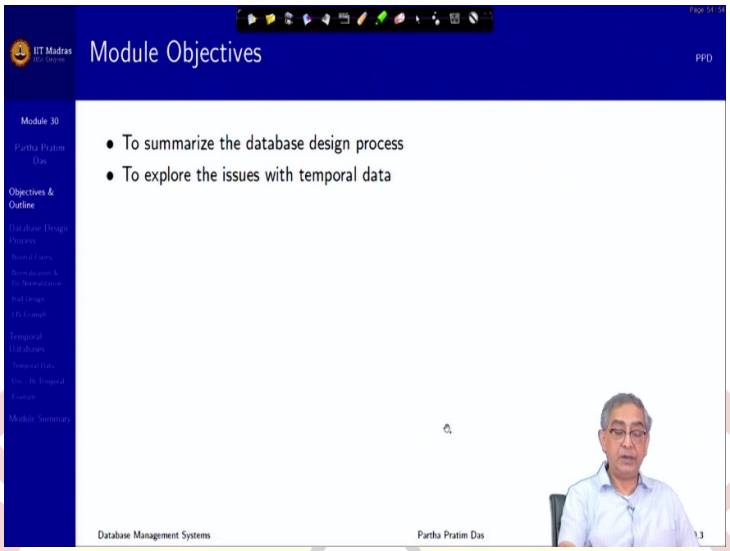
Welcome to Module 30 of database management systems course in IIT Madras online BSc program.

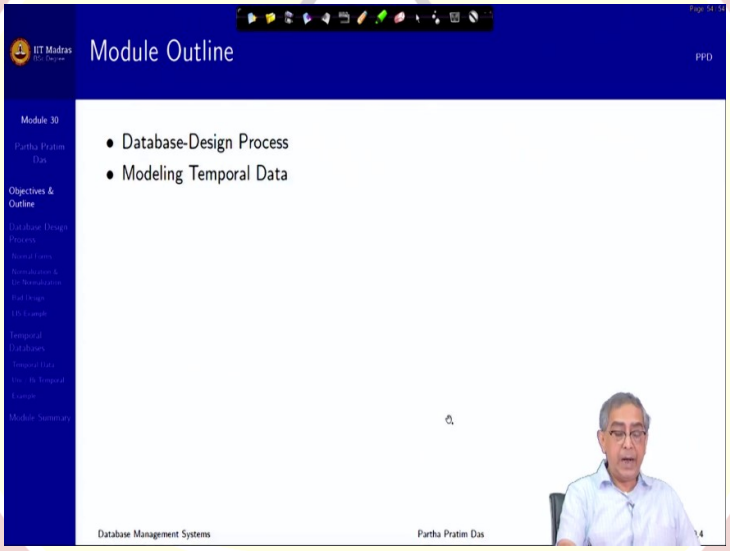
(Refer Slide Time: 00:29)



We have been discussing about Relational Database Design and we have worked through a whole lot of formal theory for that starting from functional dependency theory to normal forms that are based on functional dependency, the decomposition into normal forms the satisfaction of properties and so on. And then we have also looked at a specific situation of multiple values for attributes the multiple multivalued dependency and the consequent normal fourth normal form and decomposition into that. So, we have kind of taken a long journey through this.

(Refer Slide Time: 01:13)



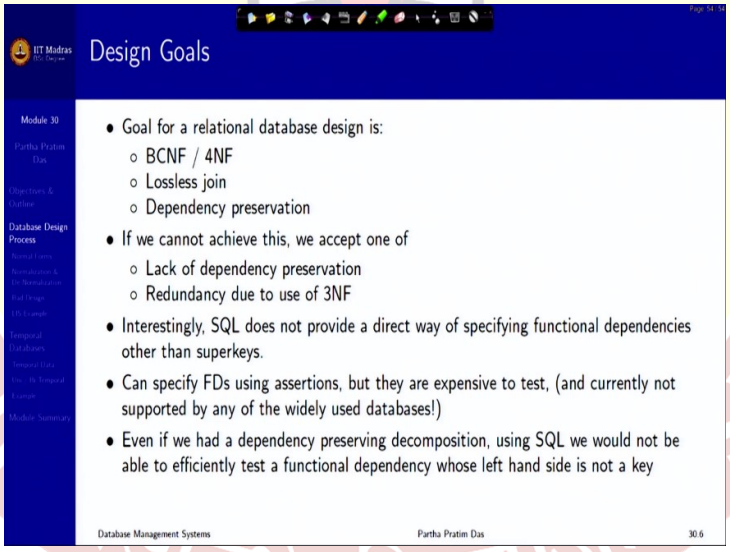


So, what we want to do in this module is to summarize the database design process.

(Refer Slide Time: 01:24)

And talk a little bit about what we have not talked at all that is in terms of what happens if time becomes a component in a database.

(Refer Slide Time: 01:35)

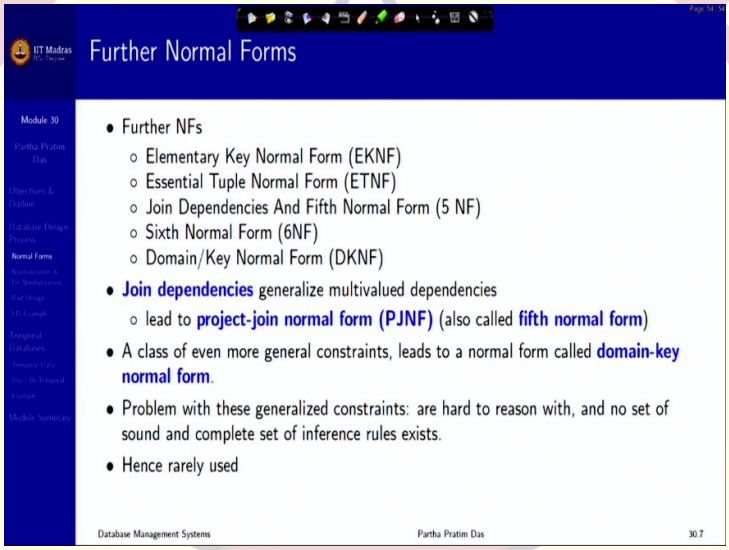


First of about the database design process just this is more a summarization actually. So, our design goals have been to do a relational database design in BCNF or 4NF with lossless join and dependency preservation if we fail to do that, we fall back on 3NF and accept the redundancy or if we can live with not checking some of the dependencies or checking them through join then we can remain in BCNF or 4NF also with dependency not being preserved.

Now, the fact of the matter is SQL actually does not provide any direct way of specifying functional dependencies. It just have specification for the superkey. So, SQL will always just guarantee the superkey condition, but not functional dependencies in general there have been proposals but nothing in that way. Because it is typically more expensive to do you can check for FDs using assertions that also is very expensive and SQL does not do it internally.

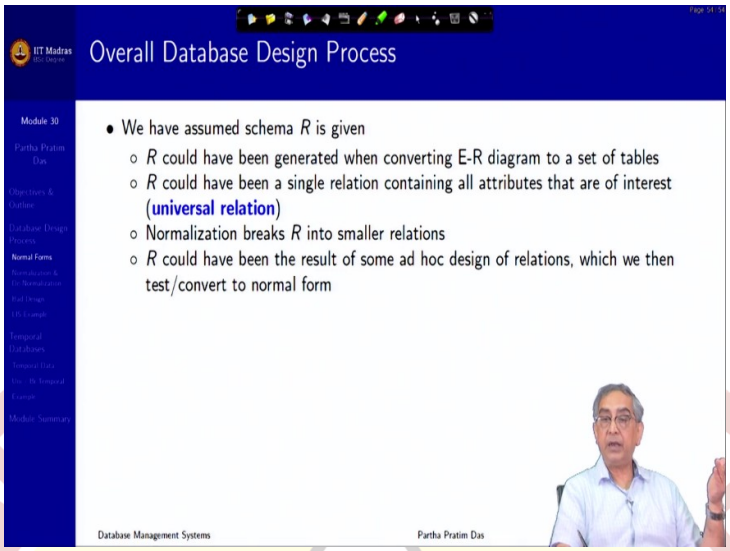
Because if for everything, you will have to check the functional dependencies then that gets very expensive in that way. So, once a design is done in the proper way, you can still do possible corruptions in that data by inserting long tuples to your application and SQL may not be able to check for that keep this in mind.

(Refer Slide Time: 03:17)



Now, as I mentioned earlier also there are various other forms of normalizations. These are some of the very people I mean, in fact, people still keep on working on newer and newer normal forms. And this is just for you to know that if you want to go deeper later in the design theory, then these are the things to learn, but we will these are rarely used in practical design. So, we are not going to discuss them.

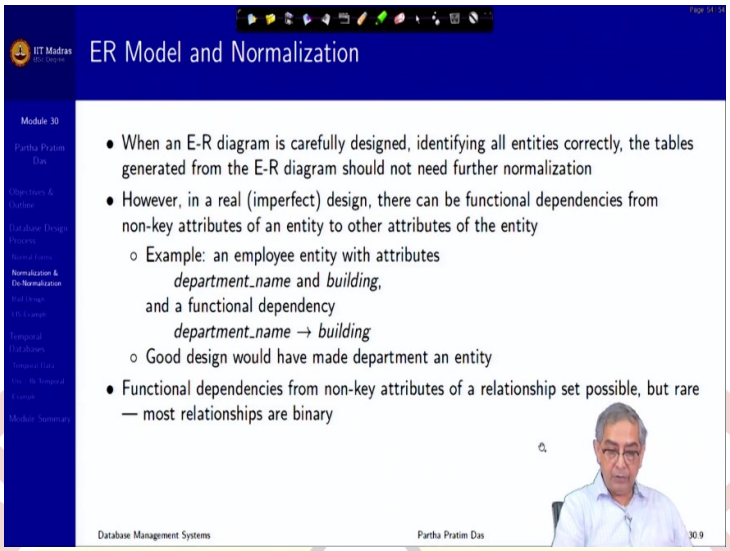
(Refer Slide Time: 03:47)



Now, if you think about the overall design process, we have also worked through a particular example for Library Information System then, we the starting point is a relational schema which could have been generated while converting the ER diagram into the set of tables like we did for the Library Information System or it could come from a single relation which had all the attributes typically called a universal relation and normalization breaks this relation into smaller relation.

It could also be that are could be some adhoc designs you know often time people do not go through either the formalism of the entire ER model they will add up say okay this table this table this table and then we have to convert them to normal form. So, wherever we start, finally the process is to convert them to the normal form, reduce data redundancy, reduce anomaly in the process and ensure better consistency in the whole thing.

(Refer Slide Time: 04:57)

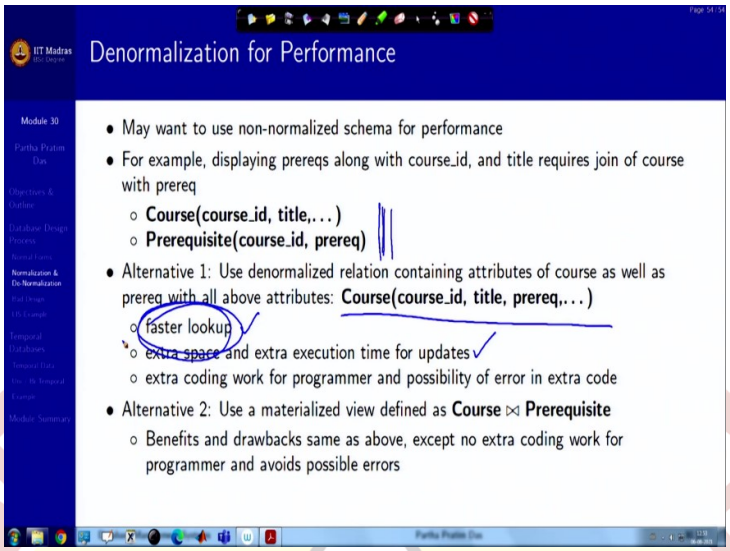


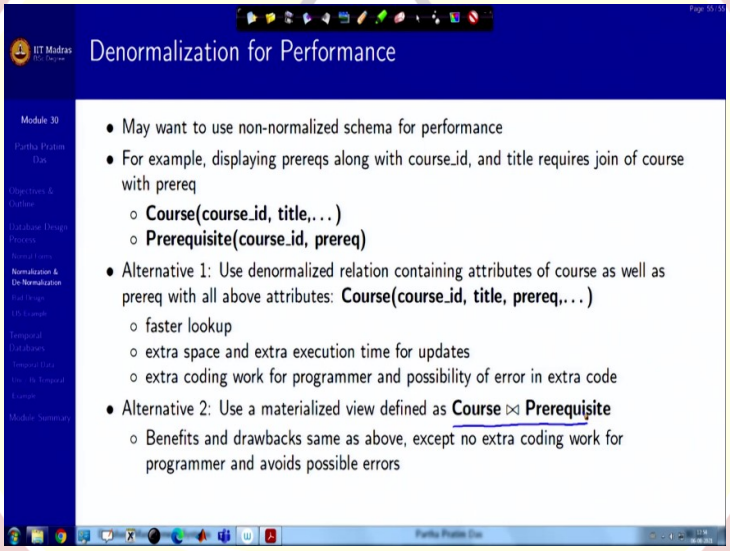
So, ER model and normalization has been our main tool. Now, in many times, there are other design factors that come in, for example, one that we saw in the member design in LIS. Another for example here is in the employee department and building kind of a relationship. The employee entity has attributes like department name and building. Now, which is required for the employee, and department name has a dependency, functional dependency on I mean, department name functionally determines the building.

But if you, we can, we can we can do this in multiple ways. But if you do a good design, then possibly you would have identified that department name should not be an attribute. Rather, it should be an attribute for an entity called department, which has a name and there. So, it is like, you take three attributes employee name, department name, building and put them together saying the employee works in this department in this building.

And then you get a non-trivial functional dependency which does not have a left-hand side, which is the key. Instead, you could conceptualize this as department name and building not being mean just, flat attributes, but part of a concept called department entity called department and attributes of those. So, these kinds of considerations need to be applied when you do the ER model.

(Refer Slide Time: 07:01)





And subsequently, when you normalize. There is another factor I would like to draw your attention to is all through the design process. I have been trying for normalization, normalize, normalize, normalize, reduce redundancy, reduce redundancy, reduce anomaly, that is that is the way to go.

But often times, what happens is, under the real-world query situation, if you have to frequently run a query, which has to do join of two relations, for example, you can think of here you have a course, which has course details and prerequisite, we saw this earlier, then, and we saw that keeping prerequisite as a part of it has, redundancy issues.

So, if you find that, then it is a choice of whether you want the best performance, or you want to live with extra storage being spent and some redundancy being around taking care of it in

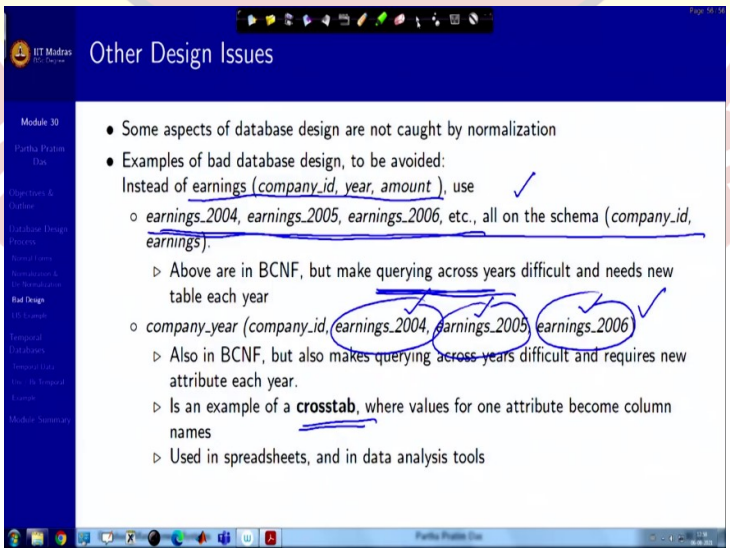
the application layer. So, if you combine it into one, then you will have a very fast lookup, because it is an index table. But if you have separate, then every time the query is filed, which possibly is several times, you have to do a joint, which is expensive.

So, it takes extra space, extra execution time for updates, because updates, particularly now we will have to do several redundant data also, and possibly extra coding work for the programmers and for the tester, but could actually improve your performance, which is a key thing. So, denormalization is also a practice that people do from time to time after having done the whole design, normalizing it getting it into shape, then people would try different queries, check the performance and might want to denormalize some.

The other way could be that you could have a materialized view of this joint. So, the benefits are almost the same. But it is just that the the developer does not need to do a lot of coding work, but the core of whatever way you decide to do, but the core idea is do not. I mean, while normalization and reduction of redundancy is very critical.

But always remember at the end that it is the ability to deliver in right time from the from every query in the database is what the customer pays for us. So, they are paying for this. So, if we if we have an excellent design, which is slow in the query execution, the customer is not going to pay. So, you will have to do balance. That is that is the design balance that you have to look at.

(Refer Slide Time: 10:05)



So, there are some other design issues. And some of these, I will talk of in the, in the later part also, for example, suppose in many times, you will find that instead of doing a design

like this, that you have earnings of a company, the company ID, year and amount, people would start making all sorts of year wise schema is one schema for 2004, one schema for 2000.

Because naturally company's earnings balance sheet, top line, bottom line, taxes, everything are bound within one year. So now, these are easily in BCNF. That is not a problem. But the problem is, querying is very difficult. Now, if you want to I mean, you want to go for IPO, and you want to make the CEO wants you to make a statement, for the last 5 years, you have a huge problem, because these are all separate tables, you cannot get the 5 years comprehensive data anywhere.

Other tendency people have is instead of having schemas, say year wise, they will start putting attributes in the same table, which are marked for different years. This is also in BCNF. And will be full of, redundant data will full of nulls, and so on. This is what is typically called cross tabbing. Cross tab, you remember tab in Excel and so on. So, where values have an attribute, these are actually values of years, they become an attribute.

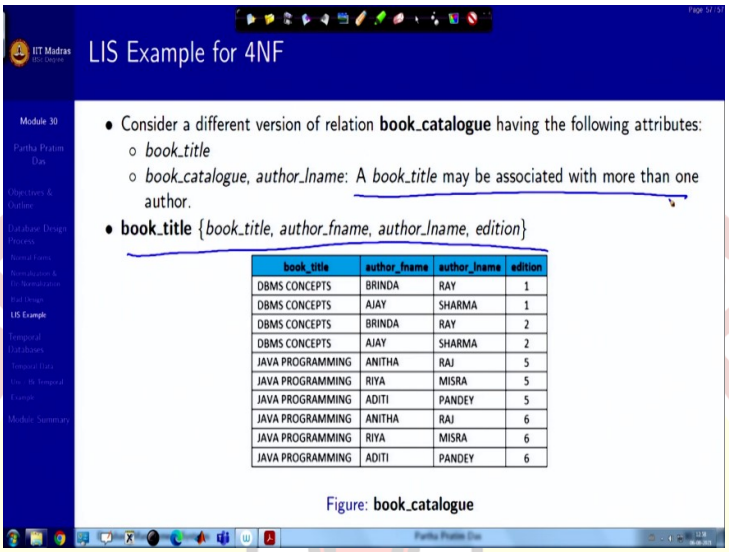
So that is called they become a column. So, this is called crosstab. So, if we, when we work on Excel, we often use this kind of technique. Remember that that is those kinds of techniques. So those kinds of strategies should not be brought in here. Certainly, a better design away from this, as well as from this is to have a year and have your data on the same table.

And all that you need is that is a filter on the year, which is can be done very, very efficiently as, as we will see, once we have certain proper indexing on the year, you will not feel the performance difference substantially. But it will be cleaner, you will be able to go to historically past data and so on. I mean, I am just talking in the perspective of historical data, this kind of issues come in, in several ways where, the core of it, this is the value as an on an ID on an attribute is meaningful for a certain period.

And then it becomes different, it is meaningful for something. For example, we were talking about the LIS Library Information System, we could have combined the student and the faculty all together in the same member table. That is it, that will not be the right thing to do. And keep that this is this is a student, but we made a proper design, so that the necessary information can go accordingly.

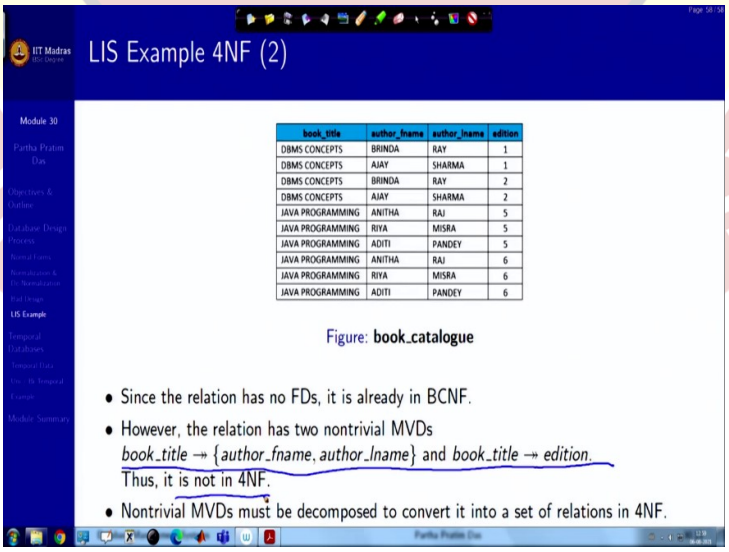
So, whenever you have multiplicity of values in terms of a certain attribute, and typically when you have a life for that, you get into these bad design issues, which you should be aware of and avoid.

(Refer Slide Time: 13:47)



Finally, I have given some extension to the LIS example, knowing that earlier we said that every book will have only one author or the first author will be used, but in general, there are multiple authors. So, there is a natural multivalued dependency in this.

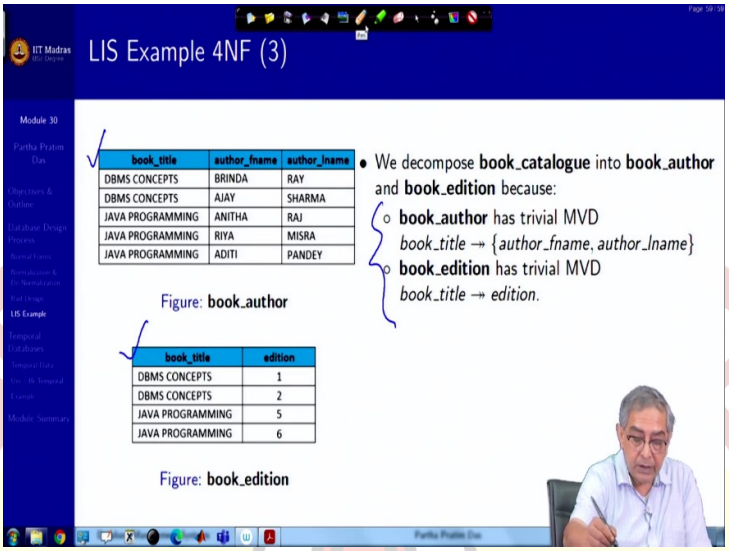
(Refer Slide Time: 14:09)



So, if you look into it, there is a multivalued dependency here. So, book, title, multi determines author name, author, fname and lname. And book title also multi determines edition because books often have multiple editions like Korths book possibly is in seventh,

eighth, ninth edition. And this is not in multivalued it is not in 4NF. So, this needs to be converted to 4NF.

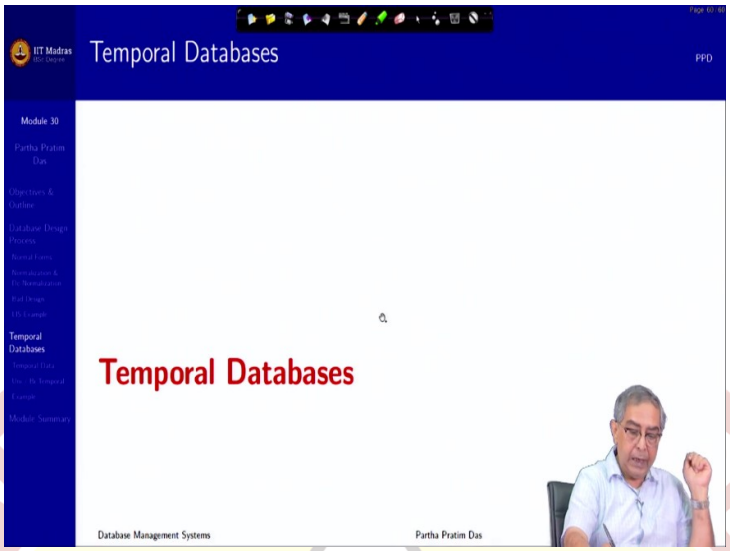
(Refer Slide Time: 14:41)



So, if you do this decomposition, you actually get to book author has trivial MVD, book title decides name and book edition, which is another so you are splitting this into two decompositions which are 4NF. So, I said that while we were doing the Library Information System example, that what we decided on a final, as a final design is not the absolute final design, you can improve on that.

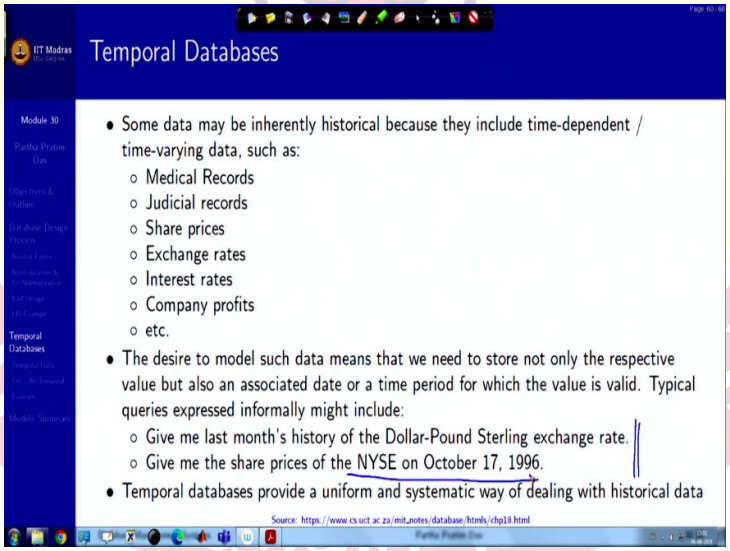
Of course, for improving, improving on that we have taken a little more flexibility in the real world that multiple authors can be maintained now, and multiple editions can be maintained now. Earlier that was not possible. So, this is these are some of the.

(Refer Slide Time: 15:38)



So, these are, this was kind of a walk around the overall design process and design issues, there will be several more. And we will come into some of those in the later modules as well. But before I close this on the design.

(Refer Slide Time: 15:58)

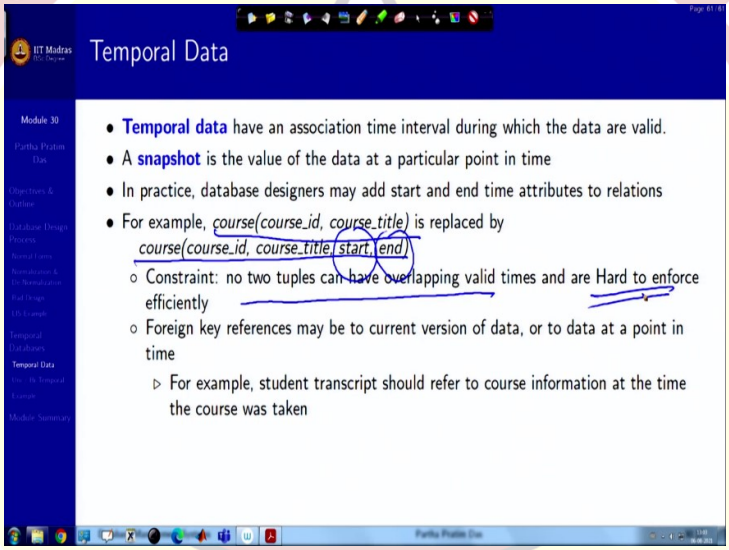


Let me talk spend a little bit of time on temporal database. See a lot of data is inherently historical. So, they are time varying. They are time dependent, for example, medical records, or medical records keep on changing, if I keep my blood pressure record, if I keep that keeps on changing with date. The database, as we have seen so far is frozen in the current time, it is a snapshot.

But typically, in several situations, the data is actually changing with time. So, it is often not enough just to update an attribute. Because you may need to know the earlier data as well. It is true for judiciary records, share prices, your stock market exchange, exchange rates interest rates, so much so much. So, we could often have queries asked in a in a way like this, give me the last month's history of the Dollar-Pound Sterling exchange rate it changes every day.

Give me the share prices on this that changes almost certain seconds or milliseconds. So, whatever design or whatever, principles we have studied so far, was not focused towards handling this kind of real-world situation. So, there are working temporal databases which try to handle the temporal data in a uniform and systematic manner.

(Refer Slide Time: 17:49)



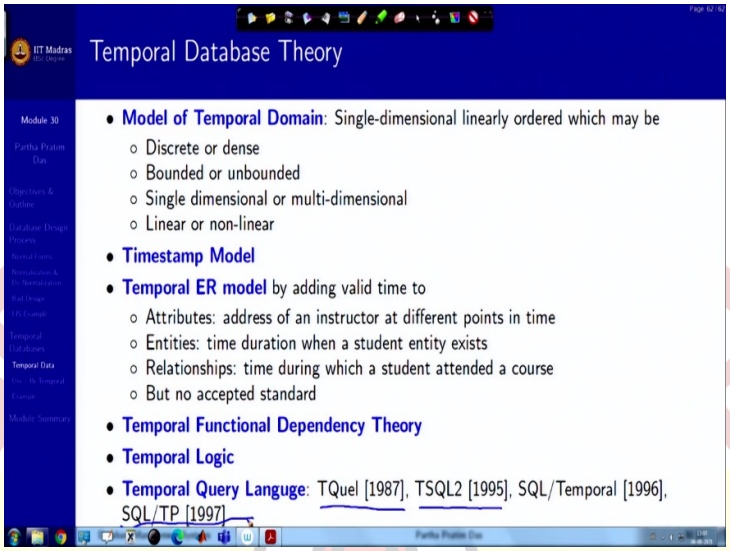
So, temporal data have an association with a time interval during which this data is valid. So, it is not only the data, but the interval, which is important. For this time to this time, this data is valid, you know is not valid afterwards. And at a particular given time, you have just a snapshot. So, just to look into the kind of examples we have done. So, you have a course, course ID, course title here.

Now, this is true for a certain semester, the next semester is different. So, it changes. So, what you might want to keep is start and end timestamps that this course ID this course title is from this date to this date, from this semester to this semester, whatever. Now, obviously, there are constraints which are difficult to enforce.

For example, if you have the start end, there is no two tuples should overlap. The same course title cannot happen in two records with overlapping time. So, it is hard to enforce. So, there

are issues with that there are several issues with that, but these are the kinds of things that are required in the real life.

(Refer Slide Time: 19:11)



So, as we have seen the evolution and are aware we have worked to certain part of the database theory design theory. So, for temporal databases, a lot of theory is required. First you need to model temporal domain you have to model time, time goes forward only does not come. So, it is a single dimensional linearly ordered. But even then, it needs it could have a different kind of modelling like it could be discrete, it could be dense, it could be bounded time.

It could be unbounded time. I could have multiple timescales, single or multi-dimensional. It could be linear. Or it could be telescopic. That when you talk about evolution of continents the time that you talk of, and when you talk about the, studies in a coursework the time that you talk of, and when you talk about change of stock prices, the time that you talk they are not on the same scale so, they are nonlinear.

One common approach one very predominant approach has been to use timestamp as just saying that you put certain timestamp on to that. Now, certainly associated questions are if you have a temporal data, then you need the ER model to be extended for temporal data, like attributes will have a lifetime address of an instructor at different points of time or an entity for which duration for a certain duration a student exists in in an institute.

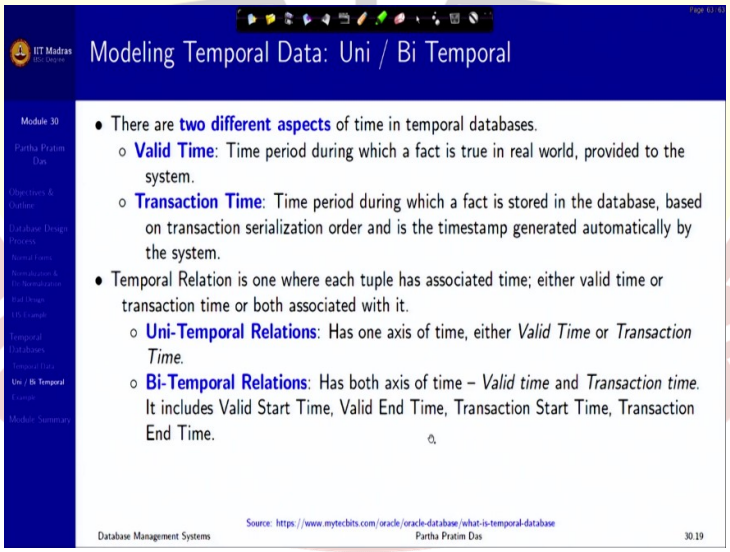
Before that, she was not a student after that she is again not a student. So, how do you capture these kinds of things in the ER model, so, there has been a lot of research into extending ER

model with time not but no standard has emerged as yet people are still working and then all the things that we have done needs that temporal extension for example, we have functional dependency theory, certainly you need temporal functional dependency this for alpha functionally determines beta at time t.

But alpha may functionally determine gamma at time t primed temporal functional dependency theory, then we have based whole of our model on as we said, predicate calculus, based on that we have developed the relational calculus, which we use for the query language and so on. So, that is the equivalent logic system for that, which is the temporal logic which is a well-developed system, not so much used in the database context, but it is a well-developed system.

So, now we have to develop the query system based on the temporal logic and create query temporal query languages, so that you can read them. So there have been quite some work in and you can see that it is it is not only recent, like it this is one of this relatively more popular SQL TP, which is in from 97, but none of them have come to a stage of really being standardized to an extent or everybody is following some or other somewhat adhoc design.

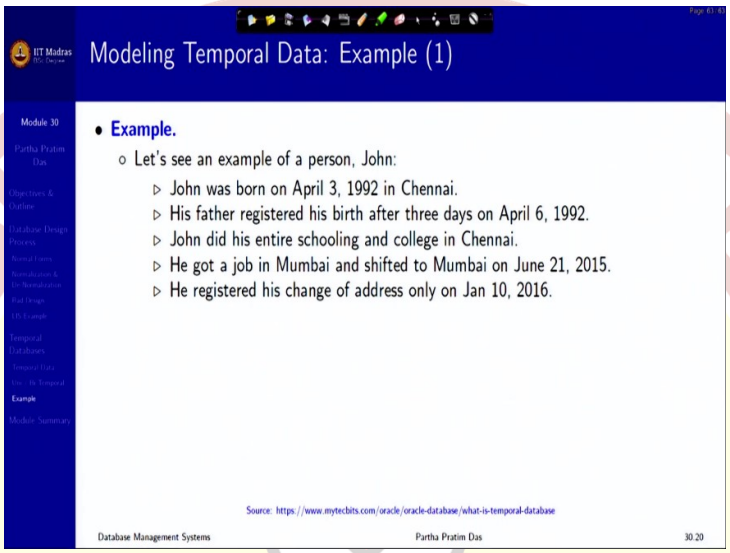
(Refer Slide Time: 22:41)



So, that is one basic design, very fundamental design strategy that people use are called look at two aspects of time. One aspect is the validity, if I talk about the value of an attribute, what is the time in the real world for which that attribute is valid, that is a valid time concept. Other is the transaction time the time period for which the value has been stored in the database.

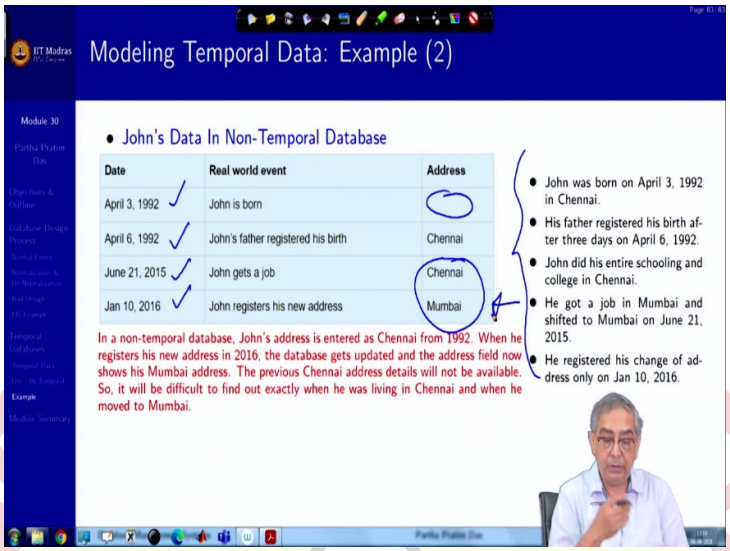
The value not necessarily has been stored in the database at the valid time. So, that is based on the transaction serialization order timestamp generated and so on. And there are typically two tracks people follow. One is called uni temporal relations, where you have either the valid time or the transaction time. The other is there could be bi temporal relation where you have valid time as well as transaction time.

(Refer Slide Time: 23:54)



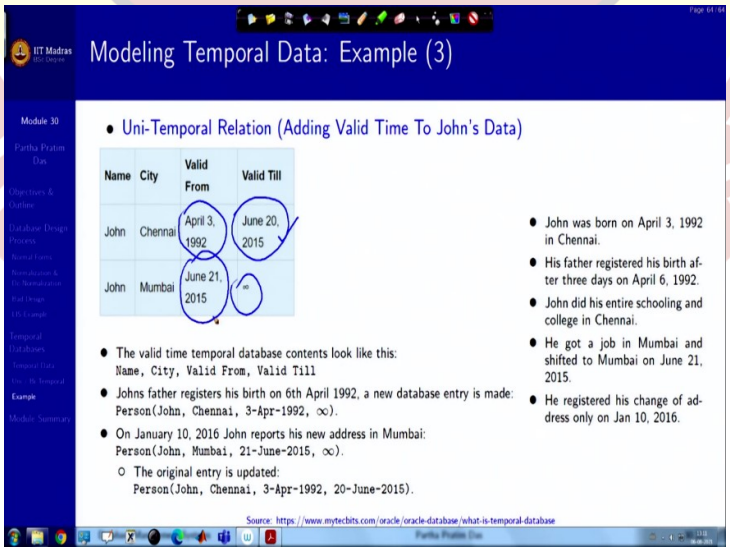
So, let us take an example. Let us say let us look at the life of a person John was born in on April 3, 1992, in Chennai. Father registered his birth of 3 days after on April 6, 1992. John did his entire schooling and college in Chennai. He got a job in Mumbai and shifted to Mumbai on June 21, 2015. And he registered his Change of Address only on January 10, 2016. So, this is this is the lifetime of the data that we are taking, this is temporary changes in the data.

(Refer Slide Time: 24:38



So how will it look? John's data in the temporary database. So, these are just just for reference, I have kept the you know, John's life history here. So, this is john is born, John's birth is registered. John gets a job listed here. Now if you see That here john does not have an address which is right because it does not registered. So, but here, you see a problem. If you look at the data now, his address is Mumbai. But, so, the previous details of the fact that he used to be in Chennai he was born in Chennai, all those get erased. So, we will have to make it make a way to keep this history in the database in a structured way.

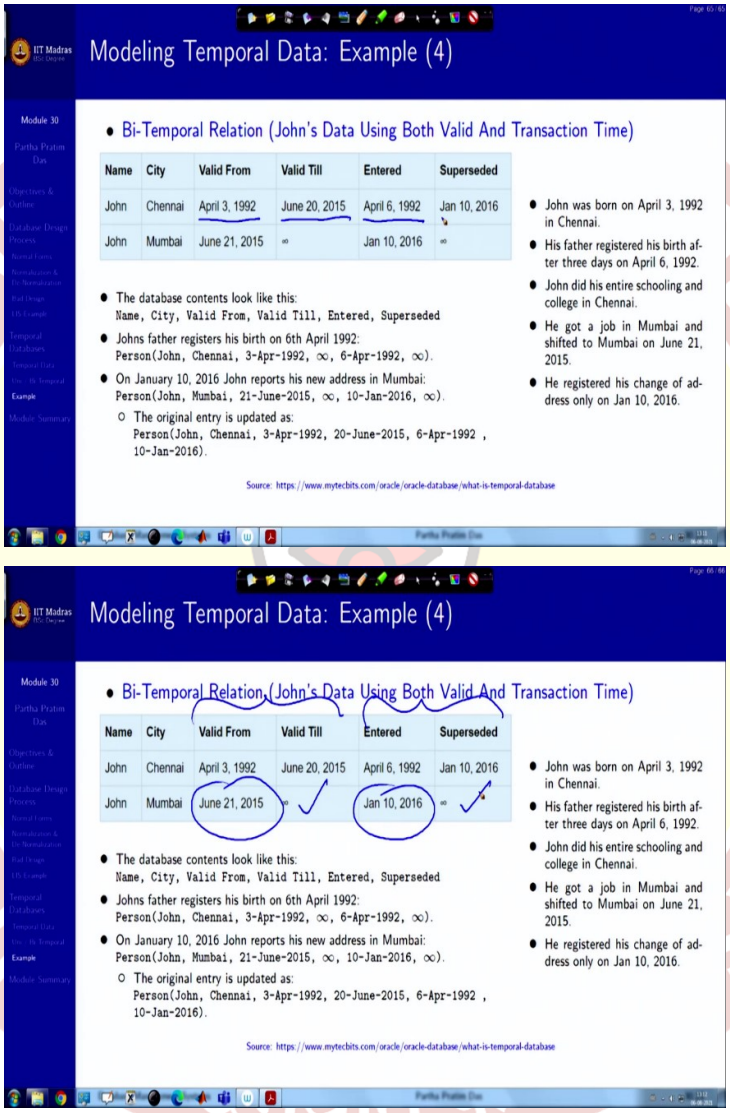
(Refer Slide Time: 25:36)



So, we attach a valid time with the data say the city of residence, we attach a valid time with that. So, we say that it is valid from the time he his birth was registered as the day he is born

to the time he changed to Mumbai and then the time is in Mumbai and it is valid till infinity as long as it goes. So, this is how if we keep this valid time then we know then you will be able to say that well where was he living on say April 2, 2014 or on October 5, 2017. So, those kinds of things will be possible to be answered.

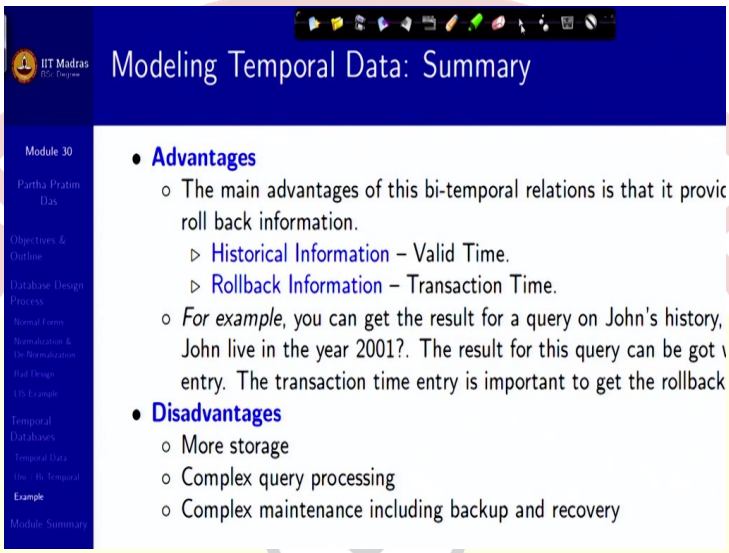
(Refer Slide Time: 26:40)



Now, the question is as the validity goes, the updates did not happen according to that. Updates happen in a different way. For example, he was born on this date, but the update happened on April 6, 1992 the address was valid till June 20, 2015. But he change, registered the change on January 10, 2016. So, this is your so, this is what I was saying this is your valid time and this is your transaction time when you first entered the data and when you say it is suspended that is it ceases to be valid anymore.

So, here the validity changes to this, but happens on a on a later date. And then validity continues and no further transaction has been done. So, this is this is just taking a very typical example of in the in the absence of a formalized approach in theory and people are having to finally model time inside the relational system itself. And that is typically an approach that people have often taken.

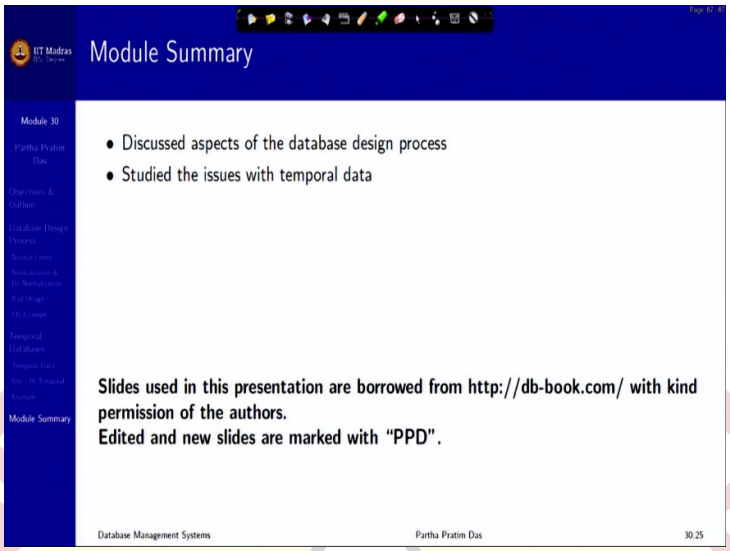
(Refer Slide Time: 27:59)



The main advantage of bitemporal relations is it provides the historical data valid time and it provides a roll back information transaction time. So, it gives you both so it makes it flexible and powerful. And you can ask a lot of historical questions on this. But certainly, it has disadvantages as well it needs more storage.

Naturally query processing gets far more complex, because you have to do a lot of filters, range checking all those and certainly maintenance particularly backup and recovery gets more complex as well. But if you need the historical data, then this is the approach that you can start using and also read up more on the status of systems and research in the temporal databases.

(Refer Slide Time: 28:54)



So, we have talked about the summarize on the database design process, particularly the relational one and introduce the basic notions of temporal data. So, with this, we are concluding on the basic design, conceptual design process of the databases and from the next module next week. We will get on with the other aspects of database management system. Thank you all very much for your attention and see you next week with the next module.